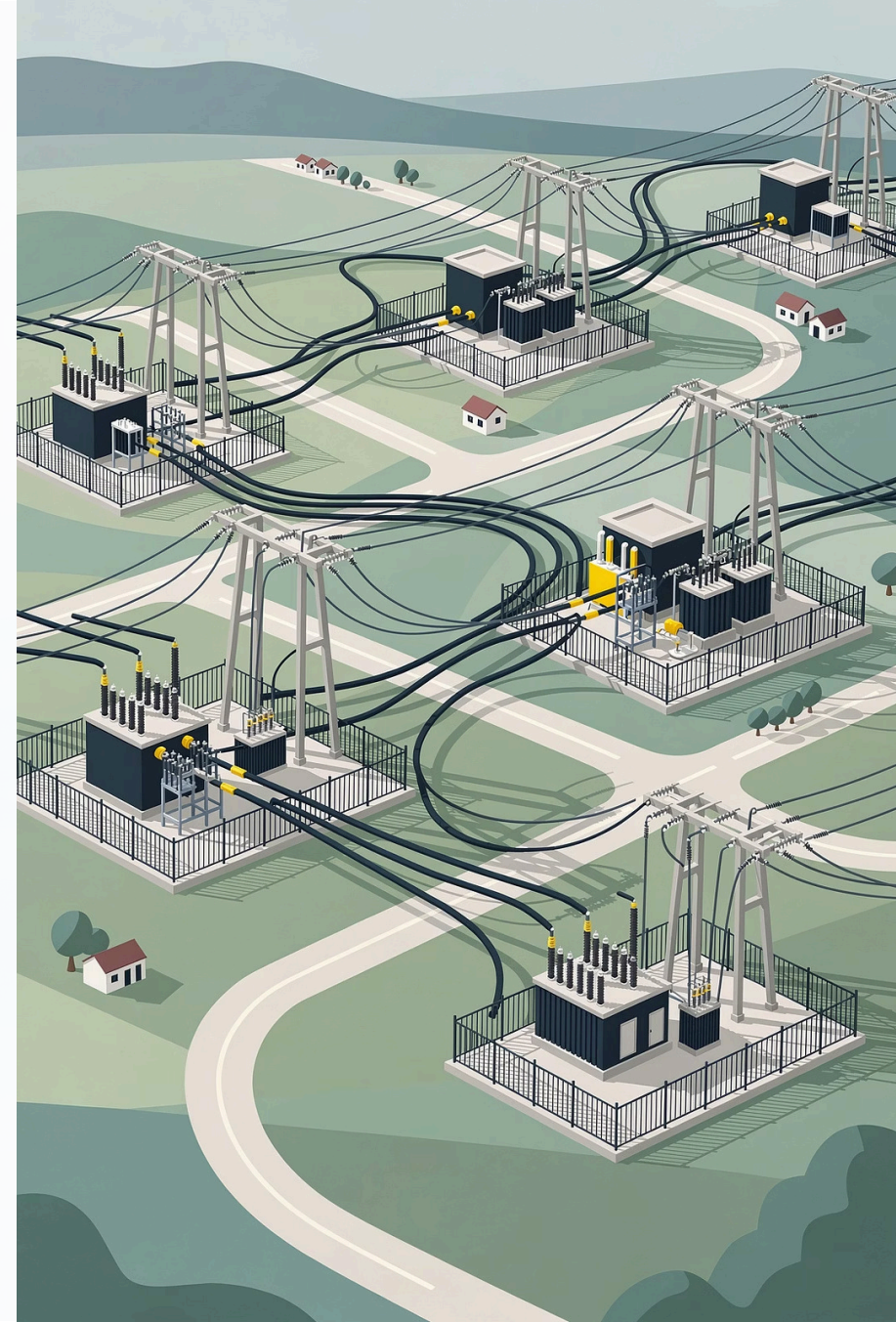


GridLink: The Utility Network Planner Hackathon

Weighted Graphs, Minimum Spanning Trees, Prim's & Kruskal's Algorithms ·
Problem Statements

DEVELOPED BY TALENCIAGLOBAL



About This Hackathon

GridLink is a utility network planner built on weighted graphs. Substations are vertices and candidate cables are undirected weighted edges; connecting every substation at minimum total cable cost is a minimum spanning tree (MST). Across six levels you will survey the grid, group stations with union-find, compute the cheapest cabling with Kruskal's algorithm, grow the network from a hub with Prim's algorithm, analyse resilience, and plan a phased rollout that reuses links already built.

Input Format

Weighted graphs are given as N M then M undirected edges u v w .
The union-find level reads operation lines (union or query).

Determinism & Difficulty

Edges are sorted by (weight, then u , then v) so the algorithms are deterministic; the MST total weight and bottleneck edge are the same for every MST. The levels run from intermediate to expert. No starter code is provided — design and write each solution yourself.

Levels

Six progressive challenges take you from basic graph surveying to expert-level phased network rollout.



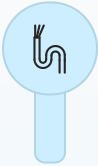
Level 1 — Survey the Grid

1



Level 2 — Group the Stations

2



Level 3 — Cheapest Cabling

3



Level 4 — Grow from the Hub

4



Level 5 — Grid Resilience

5



Level 6 — Phased Rollout

6

Level 1 — Survey the Grid

★★★ INTERMEDIATE

WEIGHTED GRAPH BASICS

Scenario: Start by surveying the candidate cabling. Read the weighted graph and report its basic figures: the number of substations and cables, the total cable weight, and the cheapest and most expensive single cable.

Concepts Covered

- Weighted undirected edges
- Summing edge weights
- Minimum and maximum edge weight
- Reading a weighted edge list

Problem Statement

Given: A line `N M`, then `M` undirected weighted edges `u v w`

Required output:

- The number of nodes
- The number of edges
- The total weight over all edges
- The minimum edge weight
- The maximum edge weight

Constraints: The graph is undirected and weighted. Vertices are labelled 0 to `N-1`. Each edge contributes its weight once to the total.

Formula Guidance

`total` = sum of all edge weights
`min edge` = smallest `w` ; `max edge` = largest `w`

Sample Input

```
5 6
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
```

Sample Output

```
Nodes: 5
Edges: 6
Total Weight: 31
Min Edge: 2
Max Edge: 8
```

 **Common Pitfall:** Count each undirected edge's weight once for the total; the minimum and maximum are over individual edge weights, not over node degrees.

 **Thinking Skill:** A weighted graph is an edge list carrying costs; before running any MST algorithm it pays to know the total, the cheapest and the dearest cable — the raw material the optimisation works on.

Level 2 — Group the Stations

★★★ INTERMEDIATE

UNION-FIND (DISJOINT SET UNION)

Scenario: Kruskal's algorithm leans on a union-find structure that tracks which stations are already linked. Process a sequence of union and query operations, answering each connectivity query and finally reporting how many separate groups remain.

Concepts Covered

- Disjoint-set union (union-find)
- find with path compression
- Union of two sets
- Counting connected groups

Problem Statement

Given: A line N T , then T operations: union a b or query a b

Required output:

- For each query: query a b = Yes/No depending on whether a and b are in the same group
- The number of groups (components) at the end

Constraints: A union merges the groups of a and b . A query reports whether a and b are already connected. Components are counted as distinct set roots.

Formula Guidance

find(x): follow parents to the root (compressing the path)

union(a, b): attach one root under the other

components = number of distinct roots

Sample Input

```
6 6
union 0 1
union 1 2
union 3 4
query 0 2
query 0 3
query 3 4
```

Sample Output

```
query 0 2 = Yes
query 0 3 = No
query 3 4 = Yes
Components: 3
```

 **Common Pitfall:** Always compare ROOTS (via find), not the raw labels; two stations can be connected through intermediaries even when their parent pointers differ.

 **Thinking Skill:** Union-find answers 'are these already connected?' in almost $O(1)$; it is the engine that lets Kruskal's algorithm reject cycle-forming edges efficiently.

Level 3 — Cheapest Cabling

★★★★ ADVANCED

KRUSKAL'S ALGORITHM

Scenario: Lay the cheapest cabling that connects every substation. Kruskal's algorithm sorts the cables by cost and adds each one that joins two so-far separate groups, skipping any that would form a loop. Report the total cost, the number of cables used, and whether the grid ends up fully connected.

Concepts Covered

- Kruskal's MST algorithm
- Sorting edges by weight
- Union-find to reject cycles
- Detecting full connectivity

Problem Statement

Given: A weighted graph $N \times M$ with M undirected edges

Required output:

- The MST total weight
- The number of cables used (MST edges)
- Whether the grid is fully connected (Yes if the tree has $N-1$ edges)

Constraints: Sort edges by (weight, u , v) for a deterministic result. Add an edge only if it joins two different groups. A connected grid uses exactly $N-1$ cables.

Formula Guidance

```
sort edges by (w, u, v)
for each edge:
    if find(u) != find(v):
        union, add weight, count++
connected = (count == N - 1)
```

Sample Input

```
5 6
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
```

Sample Output

```
MST Weight: 16
Cables: 4
Connected: Yes
```

 **Common Pitfall:** Skip an edge whose endpoints are already connected; adding it forms a cycle, inflates the cost, and breaks the spanning-tree edge count.

 **Thinking Skill:** Kruskal's algorithm is the greedy MST: take the globally cheapest edge that does not create a cycle, and union-find makes the cycle test fast — the smallest total cost emerges edge by edge.

Level 4 — Grow from the Hub

★★★★ ADVANCED

PRIM'S ALGORITHM

Scenario: Build the same network a different way: start at the hub (vertex 0) and repeatedly pull in the cheapest cable that reaches a new substation. Prim's algorithm grows one connected tree using a min-heap. Report the MST total weight and the order substations are added.

Concepts Covered

- Prim's MST algorithm
- Growing a single tree with a min-heap
- Lazy deletion of stale heap entries
- Build (insertion) order

Problem Statement

Given: A weighted graph $N \times M$ with M undirected edges

Required output:

- The MST total weight
- The order substations are added to the tree, starting from vertex 0

Constraints: Start from vertex 0. Always extend by the cheapest edge to an unvisited vertex. Adjacency lists are kept sorted for a deterministic build order.

Formula Guidance

```
push(0, source)
pop the smallest (w, u)
if u unvisited:
    add w, mark, push its edges
build order = vertices popped and added
```

Sample Input

```
5 6
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
```

Sample Output

```
MST Weight: 16
Build Order: 0 1 2 4 3
```

 **Common Pitfall:** Skip heap entries for vertices already in the tree (lazy deletion); adding a vertex twice double-counts its edge and corrupts both the weight and the build order.

 **Thinking Skill:** Prim's algorithm grows the tree outward from a seed, always taking the cheapest frontier edge — the same minimum total as Kruskal's, reached by extending one component instead of merging many.

Level 5 — Grid Resilience

★★★★ ADVANCED

MST WEIGHT, BOTTLENECK & COMPONENTS

Scenario: Analyse the planned grid. Beyond the total cost, the bottleneck — the single most expensive cable in the MST — sets the heaviest link the network depends on, and the component count reveals whether the grid is whole or split. Report all three.

Concepts Covered

- MST total weight
- The bottleneck (maximum MST edge)
- Component count from a forest
- MST edge-weight invariance

Problem Statement

Given: A weighted graph $N \times M$ with M undirected edges

Required output:

- The MST (or spanning forest) total weight
- The bottleneck: the maximum weight among the chosen edges
- The number of connected components

Constraints: Build the minimum spanning forest with Kruskal's algorithm. The bottleneck is the largest weight among chosen edges. Components = N minus the number of chosen edges.

Formula Guidance

run Kruskal, tracking:
total, chosen count, max chosen **weight**
components = N - chosen

Sample Input

6 7
0 1 2
0 3 6
1 2 3
1 4 5
2 4 7
3 4 9
4 5 4

Sample Output

MST Weight: 20
Bottleneck: 6
Components: 1

⚠

Common Pitfall: The bottleneck is the maximum of the CHOSEN (tree) edges, not of all edges; and a disconnected graph yields a forest, so report N minus the chosen-edge count as the component total.

ℹ

Thinking Skill: The multiset of edge weights in an MST is invariant, so the bottleneck (its maximum edge) is well defined regardless of which MST you build — a single number that bounds the network's weakest necessary link.

Level 6 — Phased Rollout

★★★★★ EXPERT

MST WITH PRE-BUILT LINKS (CAPSTONE)

Scenario: Finish with a phased rollout. Some substations are already linked from an earlier phase (free), so only the remaining connections cost money. Seed the union-find with the pre-built links, then run Kruskal's algorithm over the candidate cables to find the minimum additional cost to connect everything.

Concepts Covered

- MST with initial unions
- Reusing free/pre-built edges
- Kruskal over the remaining candidates
- Final connectivity check

Problem Statement

Given: A line $N \ K \ M$, then K free links $a \ b$, then M candidate edges $u \ v \ w$

Required output:

- The minimum extra cabling cost to connect everything
- The number of new cables used
- Whether the grid is fully connected afterwards

Constraints: Pre-built links merge groups at zero cost before any cabling. Then run Kruskal over the candidate edges. The grid is connected if it ends as a single component.

Formula Guidance

union the K free links first (cost 0)
run Kruskal over the M candidate edges,
adding only cross-group edges
connected = (one component remains)

Sample Input

```
5 1 6
0 1
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
```

Sample Output

Extra Cost: 14
Cables Used: 3
Connected: Yes

 **Common Pitfall:** Apply the free links to union-find BEFORE the cabling pass; processing them afterwards (or charging for them) changes which candidate edges are still needed and the extra cost.

 **Thinking Skill:** Pre-merging the free links before running Kruskal generalises the MST to a Steiner-like 'connect the rest cheaply' problem — the union-find simply starts from a partial solution instead of singletons.